

hwk2/tb3_logic_shift.vhd

```

1  -----
2  -- TB3_LogicShift  --  SH-2 CPU Integration Testbench 3
3  --
4  -- Two phases controlled by a single signal "loading":
5  --   loading='1' : TB owns the bus (load hex, then check memory)
6  --                   CPU is held in reset so it tri-states all its outputs
7  --   loading='0' : CPU owns the bus (program execution)
8  --                   TB drives 'Z' so it doesn't interfere
9  --
10 -- CPU, memory, and TB all share the same resolved bus signals.
11 -- Only one non-Z driver is active at any given time.
12 --
13 -- Instructions covered:
14 --   AND Rm,Rn   OR Rm,Rn   XOR Rm,Rn   NOT Rm,Rn   TST Rm,Rn
15 --   AND #imm,R0   OR #imm,R0
16 --   SHLL  SHLR   SHLL16  SHLR16
17 --   SHAL  SHAR   ROTL    ROTCL  ROTCR
18 --   CLRT  MOVT
19 --
20 -- Expected results in data memory (block 1):
21 --   0x1000  0x00000000  AND F0F0&0F0F=0
22 --   0x1004  0xFFFFFFFF  OR  F0F0|0F0F=FFFF
23 --   0x1008  0xFFFFFFFF  XOR AAAA^5555=FFFF
24 --   0x100C  0x55555555  NOT ~0xAAAAAAAA
25 --   0x1010  0x00000001  TST zero result -> T=1
26 --   0x1014  0x0000000F  AND#imm 0xFF & 0x0F
27 --   0x1018  0x000000A5  OR#imm 0x00 | 0xA5
28 --   0x101C  0x00000002  SHLL 0x80000001
29 --   0x1020  0x00000001  T after SHLL
30 --   0x1024  0x40000000  SHLR 0x80000001
31 --   0x1028  0x00000001  T after SHLR
32 --   0x102C  0xABCD0000  SHLL16 0x0000ABCD
33 --   0x1030  0x0000ABCD  SHLR16 result
34 --   0x1034  0x80000002  SHAL 0x40000001
35 --   0x1038  0x00000000  T after SHAL (MSB was 0)
36 --   0x103C  0xC0000001  SHAR 0x80000002
37 --   0x1040  0x00000003  ROTL 0x80000001
38 --   0x1044  0x00000001  T after ROTL
39 --   0x1048  0x55555554  ROTCL 0xAAAAAAAA T=0
40 --   0x104C  0x00000001  T after ROTCL
41 --   0x1050  0xAAAAAAAA  ROTCR 0x55555554 T=1
42 --   0x1054  0x00000000  T after ROTCR
43 --
44 -- Program image (assembled; 53 words = 212 bytes):
45 --
46 -- Disassembly (address : encoding : instruction):
47 -- -- Setup --
48 --   0x0000: DE2D  MOV.L @(0x2D,PC),R14      R14 = 0x00001000
49 --   0x0002: 0009  NOP
50 -- -- Test 1: AND Rm,Rn --
51 --   0x0004: D12D  MOV.L @(0x2D,PC),R1      R1 = 0xF0F0F0F0
52 --   0x0006: D22E  MOV.L @(0x2E,PC),R2      R2 = 0x0F0F0F0F
53 --   0x0008: 2219  AND R1,R2              R2 = 0x00000000
54 --   0x000A: 2E22  MOV.L R2,@R14          [0x1000] = 0x00000000
55 --   0x000C: 7E04  ADD #4,R14
56 -- -- Test 2: OR Rm,Rn --
57 --   0x000E: D12B  MOV.L @(0x2B,PC),R1      R1 = 0xF0F0F0F0
58 --   0x0010: D22B  MOV.L @(0x2B,PC),R2      R2 = 0x0F0F0F0F
59 --   0x0012: 221B  OR R1,R2                R2 = 0xFFFFFFFF
60 --   0x0014: 2E22  MOV.L R2,@R14          [0x1004] = 0xFFFFFFFF
61 --   0x0016: 7E04  ADD #4,R14
62 -- -- Test 3: XOR Rm,Rn --
63 --   0x0018: D12A  MOV.L @(0x2A,PC),R1      R1 = 0xAAAAAAAA
64 --   0x001A: D22B  MOV.L @(0x2B,PC),R2      R2 = 0x55555555
65 --   0x001C: 221A  XOR R1,R2              R2 = 0xFFFFFFFF

```

```

66 --      0x001E: 2E22 MOV.L R2,@R14      [0x1008] = 0xFFFFFFFF
67 --      0x0020: 7E04 ADD #4,R14
68 -- -- Test 4: NOT --
69 --      0x0022: D128 MOV.L @(0x28,PC),R1  R1 = 0xAAAAAAAA
70 --      0x0024: 6217 NOT R1,R2           R2 = 0x55555555
71 --      0x0026: 2E22 MOV.L R2,@R14      [0x100C] = 0x55555555
72 --      0x0028: 7E04 ADD #4,R14
73 -- -- Test 5: TST --
74 --      0x002A: D124 MOV.L @(0x24,PC),R1  R1 = 0xF0F0F0F0
75 --      0x002C: D224 MOV.L @(0x24,PC),R2  R2 = 0xF0F0F0F0
76 --      0x002E: 2128 TST R2,R1           T = (R1 & R2 == 0) = 1
77 --      0x0030: 0329 MOVT R3             R3 = T = 1
78 --      0x0032: 2E32 MOV.L R3,@R14      [0x1010] = 0x00000001
79 --      0x0034: 7E04 ADD #4,R14
80 -- -- Test 6: AND #imm,R0 --
81 --      0x0036: E0FF MOV #-1,R0          R0 = 0xFFFFFFFF
82 --      0x0038: C90F AND #0x0F,R0       R0 = 0x0000000F
83 --      0x003A: 2E02 MOV.L R0,@R14      [0x1014] = 0x0000000F
84 --      0x003C: 7E04 ADD #4,R14
85 -- -- Test 7: OR #imm,R0 --
86 --      0x003E: E000 MOV #0,R0           R0 = 0x00000000
87 --      0x0040: CBA5 OR #0xA5,R0        R0 = 0x000000A5
88 --      0x0042: 2E02 MOV.L R0,@R14      [0x1018] = 0x000000A5
89 --      0x0044: 7E04 ADD #4,R14
90 -- -- Test 8: SHLL --
91 --      0x0046: D121 MOV.L @(0x21,PC),R1  R1 = 0x80000001
92 --      0x0048: 4100 SHLL R1             R1 = 0x00000002, T = 1
93 --      0x004A: 2E12 MOV.L R1,@R14      [0x101C] = 0x00000002
94 --      0x004C: 7E04 ADD #4,R14
95 -- -- Test 9: T after SHLL --
96 --      0x004E: 0329 MOVT R3             R3 = T = 1
97 --      0x0050: 2E32 MOV.L R3,@R14      [0x1020] = 0x00000001
98 --      0x0052: 7E04 ADD #4,R14
99 -- -- Test 10: SHLR --
100 --     0x0054: D11D MOV.L @(0x1D,PC),R1  R1 = 0x80000001
101 --     0x0056: 4101 SHLR R1             R1 = 0x40000000, T = 1
102 --     0x0058: 2E12 MOV.L R1,@R14      [0x1024] = 0x40000000
103 --     0x005A: 7E04 ADD #4,R14
104 -- -- Test 11: T after SHLR --
105 --     0x005C: 0329 MOVT R3             R3 = T = 1
106 --     0x005E: 2E32 MOV.L R3,@R14      [0x1028] = 0x00000001
107 --     0x0060: 7E04 ADD #4,R14
108 -- -- Test 12: SHLL16 --
109 --     0x0062: D11B MOV.L @(0x1B,PC),R1  R1 = 0x0000ABCD
110 --     0x0064: 4128 SHLL16 R1          R1 = 0xABCD0000
111 --     0x0066: 2E12 MOV.L R1,@R14      [0x102C] = 0xABCD0000
112 --     0x0068: 7E04 ADD #4,R14
113 -- -- Test 13: SHLR16 --
114 --     0x006A: 4129 SHLR16 R1          R1 = 0x0000ABCD
115 --     0x006C: 2E12 MOV.L R1,@R14      [0x1030] = 0x0000ABCD
116 --     0x006E: 7E04 ADD #4,R14
117 -- -- Test 14: SHAL --
118 --     0x0070: D118 MOV.L @(0x18,PC),R1  R1 = 0x40000001
119 --     0x0072: 4120 SHAL R1           R1 = 0x80000002, T = 0
120 --     0x0074: 2E12 MOV.L R1,@R14      [0x1034] = 0x80000002
121 --     0x0076: 7E04 ADD #4,R14
122 -- -- Test 15: T after SHAL --
123 --     0x0078: 0329 MOVT R3             R3 = T = 0
124 --     0x007A: 2E32 MOV.L R3,@R14      [0x1038] = 0x00000000
125 --     0x007C: 7E04 ADD #4,R14
126 -- -- Test 16: SHAR --
127 --     0x007E: 4121 SHAR R1           R1 = 0xC0000001, T = 0
128 --     0x0080: 2E12 MOV.L R1,@R14      [0x103C] = 0xC0000001
129 --     0x0082: 7E04 ADD #4,R14
130 -- -- Test 17: ROTL --
131 --     0x0084: D111 MOV.L @(0x11,PC),R1  R1 = 0x80000001
132 --     0x0086: 4104 ROTL R1           R1 = 0x00000003, T = 1

```

```

133 --      0x0088: 2E12  MOV.L R1,@R14          [0x1040] = 0x00000003
134 --      0x008A: 7E04  ADD #4,R14
135 -- -- Test 18: T after ROTL --
136 --      0x008C: 0329  MOV.T R3                R3 = T = 1
137 --      0x008E: 2E32  MOV.L R3,@R14          [0x1044] = 0x00000001
138 --      0x0090: 7E04  ADD #4,R14
139 -- -- Test 19: ROTCL --
140 --      0x0092: 0008  CLRT                    T = 0
141 --      0x0094: D10B  MOV.L @(0x0B,PC),R1      R1 = 0xAAAAAAAA
142 --      0x0096: 4124  ROTCL R1                R1 = 0x55555554, T = 1
143 --      0x0098: 2E12  MOV.L R1,@R14          [0x1048] = 0x55555554
144 --      0x009A: 7E04  ADD #4,R14
145 -- -- Test 20: T after ROTCL --
146 --      0x009C: 0329  MOV.T R3                R3 = T = 1
147 --      0x009E: 2E32  MOV.L R3,@R14          [0x104C] = 0x00000001
148 --      0x00A0: 7E04  ADD #4,R14
149 -- -- Test 21: ROTCR --
150 --      0x00A2: 4125  ROTCR R1                R1 = 0xAAAAAAAA, T = 0
151 --      0x00A4: 2E12  MOV.L R1,@R14          [0x1050] = 0xAAAAAAAA
152 --      0x00A6: 7E04  ADD #4,R14
153 -- -- Test 22: T after ROTCR --
154 --      0x00A8: 0329  MOV.T R3                R3 = T = 0
155 --      0x00AA: 2E32  MOV.L R3,@R14          [0x1054] = 0x00000000
156 --      0x00AC: 7E04  ADD #4,R14
157 -- -- End --
158 --      0x00AE: A000  BRA .                    self-loop
159 --      0x00B0: 0009  NOP
160 --      0x00B2: 0009  NOP
161 -- -- Data constants --
162 --      0x00B4: 00001000
163 --      0x00B8: F0F0F0F0
164 --      0x00BC: 0F0F0F0F
165 --      0x00C0: AAAAAAAAAA
166 --      0x00C4: 55555555
167 --      0x00C8: 80000001
168 --      0x00CC: 0000ABCD
169 --      0x00D0: 40000001

```

```

170 -----
171
172 library ieee;
173 use ieee.std_logic_1164.all;
174 use ieee.numeric_std.all;
175 use std.textio.all;
176
177 entity TB3_LogicShift is
178 end entity;
179
180 architecture sim of TB3_LogicShift is
181
182     constant CLK_PERIOD : time      := 20 ns;
183     constant PROG_HEX   : string    := "tb3_program.hex";
184     constant MEM_WORDS  : integer   := 256;
185
186     signal clk          : std_logic := '0';
187     signal reset        : std_logic := '0';
188     signal loading      : std_logic := '1';
189
190     -- Shared bus - CPU, memory, and TB all connect here
191     signal AB : std_logic_vector(31 downto 0);
192     signal DB : std_logic_vector(31 downto 0);
193     signal RE0, RE1, RE2, RE3 : std_logic;
194     signal WE0, WE1, WE2, WE3 : std_logic;
195
196     -- TB-side bus signals
197     signal tb_AB : std_logic_vector(31 downto 0) := (others => '0');
198     signal tb_DB : std_logic_vector(31 downto 0) := (others => '0');
199     signal tb_RE : std_logic := '1';

```

```

200     signal tb_WE : std_logic := '1';
201
202 begin
203
204     clk <= not clk after CLK_PERIOD / 2;
205
206     -- TB bus mux: drives bus when loading='1', releases ('Z') when loading='0'
207     -- During loading='0' the CPU drives the bus instead (via its port map)
208     AB <= tb_AB when loading = '1' else (others => 'Z');
209     DB <= tb_DB when (loading = '1' and tb_WE = '0') else (others => 'Z');
210     RE0 <= tb_RE when loading = '1' else 'Z';
211     RE1 <= tb_RE when loading = '1' else 'Z';
212     RE2 <= tb_RE when loading = '1' else 'Z';
213     RE3 <= tb_RE when loading = '1' else 'Z';
214     WE0 <= tb_WE when loading = '1' else 'Z';
215     WE1 <= tb_WE when loading = '1' else 'Z';
216     WE2 <= tb_WE when loading = '1' else 'Z';
217     WE3 <= tb_WE when loading = '1' else 'Z';
218
219     -- CPU connects directly to the shared bus
220     dut : entity work.SH2_CPU
221         port map (
222             Reset => reset, NMI => '1', INT => '1', clock => clk,
223             AB => AB, DB => DB,
224             RE0 => RE0, RE1 => RE1, RE2 => RE2, RE3 => RE3,
225             WE0 => WE0, WE1 => WE1, WE2 => WE2, WE3 => WE3
226         );
227
228     mem : entity work.MEMORY32x32
229         generic map (
230             MEMSIZE => MEM_WORDS,
231             START_ADDR0 => 16#00000000#,
232             START_ADDR1 => 16#00001000#,
233             START_ADDR2 => 16#10000000#,
234             START_ADDR3 => 16#20000000#
235         )
236         port map (
237             RE0 => RE0, RE1 => RE1, RE2 => RE2, RE3 => RE3,
238             WE0 => WE0, WE1 => WE1, WE2 => WE2, WE3 => WE3,
239             MemAB => AB, MemDB => DB
240         );
241
242     stim : process
243         file      hex_file : text;
244         variable fstatus  : file_open_status;
245         variable l        : line;
246         variable word     : std_logic_vector(31 downto 0);
247         variable addr     : unsigned(31 downto 0);
248         variable pass_cnt : integer := 0;
249         variable fail_cnt : integer := 0;
250
251         procedure check(a : std_logic_vector(31 downto 0);
252             exp : std_logic_vector(31 downto 0);
253             tag : string) is
254             begin
255                 tb_AB <= a;
256                 tb_RE <= '0';
257                 tb_WE <= '1';
258                 wait for CLK_PERIOD;
259                 if DB = exp then
260                     pass_cnt := pass_cnt + 1;
261                     report "PASS: " & tag severity note;
262                 else
263                     fail_cnt := fail_cnt + 1;
264                     report "FAIL: " & tag
265                         & " got=0x" & to_hstring(DB)
266                         & " exp=0x" & to_hstring(exp) severity error;

```

```

267         end if;
268         tb_RE <= '1';
269         wait for CLK_PERIOD;
270     end procedure;
271
272 begin
273     -----
274     -- LOAD PHASE
275     -----
276     loading <= '1';
277     reset   <= '0';
278     tb_RE   <= '1';
279     tb_WE   <= '1';
280     wait for CLK_PERIOD * 2;
281
282     file_open(fstatus, hex_file, PROG_HEX, read_mode);
283     assert fstatus = open_ok
284         report "Cannot open " & PROG_HEX severity failure;
285
286     addr := (others => '0');
287     while not endfile(hex_file) loop
288         readline(hex_file, l);
289         if l'length > 0 then
290             hread(l, word);
291             tb_AB <= std_logic_vector(addr);
292             tb_DB <= word;
293             tb_WE <= '0';
294             wait for CLK_PERIOD;
295             tb_WE <= '1';
296             wait for CLK_PERIOD;
297             addr := addr + 4;
298         end if;
299     end loop;
300     file_close(hex_file);
301
302     -----
303     -- CPU PHASE
304     -----
305     wait until rising_edge(clk);
306     loading <= '0';
307     reset   <= '1';
308
309     wait for CLK_PERIOD * 2;
310
311     for i in 1 to 500 loop
312         wait until rising_edge(clk);
313     end loop;
314
315     -----
316     -- CHECK PHASE
317     -----
318     wait until rising_edge(clk);
319     reset <= '0';
320     loading <= '1';
321     tb_RE <= '1';
322     tb_WE <= '1';
323     wait for CLK_PERIOD * 2;
324
325     check(x"00001000", x"00000000", "AND F0F0&0F0F=0");
326     check(x"00001004", x"FFFFFFFF", "OR  F0F0|0F0F=FFFF");
327     check(x"00001008", x"FFFFFFFF", "XOR AAAA^5555=FFFF");
328     check(x"0000100C", x"55555555", "NOT ~0xAAAAAAAA");
329     check(x"00001010", x"00000001", "TST zero result -> T=1");
330     check(x"00001014", x"0000000F", "AND#imm upper24 cleared");
331     check(x"00001018", x"000000A5", "OR#imm 0x00|0xA5");
332     check(x"0000101C", x"00000002", "SHLL 0x80000001");
333     check(x"00001020", x"00000001", "T after SHLL");

```

```

334     check(x"00001024", x"40000000", "SHLR 0x80000001");
335     check(x"00001028", x"00000001", "T after SHLR");
336     check(x"0000102C", x"ABCD0000", "SHLL16 0x0000ABCD");
337     check(x"00001030", x"0000ABCD", "SHLR16 result");
338     check(x"00001034", x"80000002", "SHAL 0x40000001");
339     check(x"00001038", x"00000000", "T after SHAL (MSB=0)");
340     check(x"0000103C", x"C0000001", "SHAR 0x80000002");
341     check(x"00001040", x"00000003", "ROTL 0x80000001");
342     check(x"00001044", x"00000001", "T after ROTL");
343     check(x"00001048", x"55555554", "ROTCL 0xAAAAAAAA T=0");
344     check(x"0000104C", x"00000001", "T after ROTCL");
345     check(x"00001050", x"AAAAAAAA", "ROTCR 0x55555554 T=1");
346     check(x"00001054", x"00000000", "T after ROTCR");
347
348     -----
349     -- SUMMARY
350     -----
351     report "TB3: PASSED=" & integer'image(pass_cnt)
352           & "   FAILED=" & integer'image(fail_cnt) severity note;
353     if fail_cnt = 0 then
354         report "*** ALL TESTS PASSED ***" severity note;
355     else
356         assert false
357           report integer'image(fail_cnt) & " TEST(S) FAILED"
358           severity failure;
359     end if;
360     wait;
361 end process;
362
363 end architecture;
364

```